

Iskanje nove fizike z variacijskimi avtoenkoderji

7. sklop nalog

Blaž Bortolato

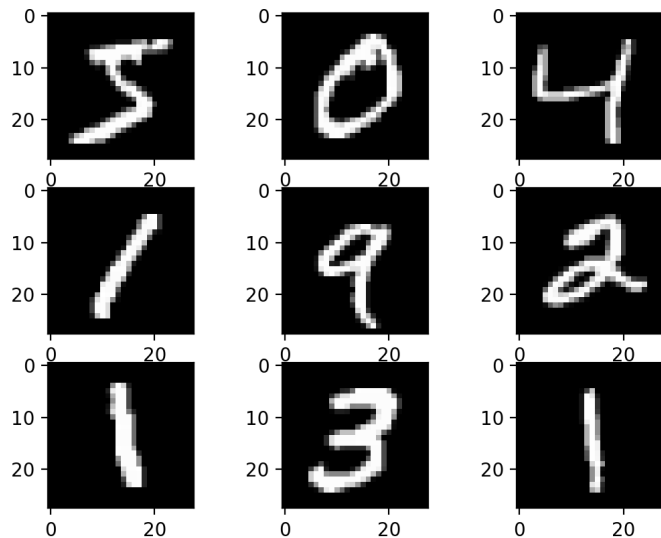
16.1.2026

7. sklop nalog

1. Gručenje MNIST z binarno mešanico
2. VAE na MNIST podatkih
3. VAE na simuliranih podatkih dvocurkovnega signala

Podatki MNIST

MNIST: množica slik ročno napisanih števk velikosti 28×28 pikslov



↓
niz s 784 števki med 0 in 255

↓
normaliziramo vrednosti med 0 in 1

1. del: Binarna mešanica in MNIST

4/29

Gručenje z binarno mešanico:

$$\mathcal{P}(x) \equiv \sum_{j=1}^{10} \frac{P(x|\mu^j)}{\pi_j} \pi_j$$

niz 784 števil za vsak j
(povprečna slika j -te gruče)

delež števk v j -ti gruči

x niz 784 števil za vsako sliko

produkt Bernoullijevih porazdelitev

$$P(x|\mu) = \prod_{i=1}^{784} \mu_i^{x_i} (1 - \mu_i)^{(1-x_i)}$$

(vrednosti so omejene na interval $[0,1]$)

1. del: Binarna mešanica in MNIST

5/29

Algorithm za določitev deležev slik $\pi_1, \dots, \pi_{10}$ in "povprečnih slik" $\mu^1, \dots, \mu^{10}$:

1. Izberi začetne vrednosti za $\pi_1, \dots, \pi_{10}$ in $\mu^1, \dots, \mu^{10}$
2. Ponavljaj:

→ Izračunaj verjetnost $P(x_l|j)$, da slika x_l pripada gruči j :

$$P(x_l|j) = \frac{P(x_l|\mu^j)}{\mathcal{P}(x_l)}$$

→ Izračunaj nove vrednosti deležev slik in "povprečne slike":

$$\mu^j = \frac{\sum_x P(x|j)x}{\sum_x P(x|j)} \qquad \pi_j = \frac{\sum_x P(x|j)}{\sum_{j=1}^{10} \sum_x P(x|j)}$$

1. del: Binarna mešanica in MNIST

6/29

1. naloga

1. Implementiraj algoritem binarnih mešanic in ga preizkusi na podatkih MNIST. Nariši slike števil, ki so predstavljene z nizi μ^j , primer kode:

```
plt.figure()  
plt.imshow(mu[j].reshape((28,28)))  
plt.show()
```

Nato poskusi generirati nekaj novih slik tako, da si izbereš (dovolj 1) gručo j in naključno izžrebaš nekaj slik iz porazdelitve $P(x|\mu^j)$, primer kode:

```
x_new = np.random.binomial(n=1, p = mu[j]).
```

poskusi različne n

2. del: VAE + MNIST

2. naloga

Implementiraj VAE po navodilih in ga zaženi na podatkih MNIST z **adam** optimizatorjem.

```
import keras

(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
```

```
In [2]: x_train.shape
Out[2]: (60000, 28, 28)
```

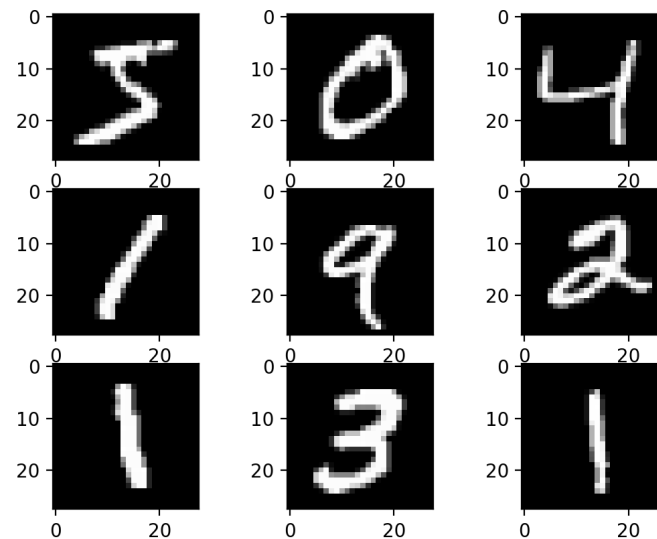
1 slika $\sim$ 784 parametrov

```
In [4]: x_train = x_train.reshape(-1, 28*28)
```

```
In [5]: x_train.shape
Out[5]: (60000, 784)
```

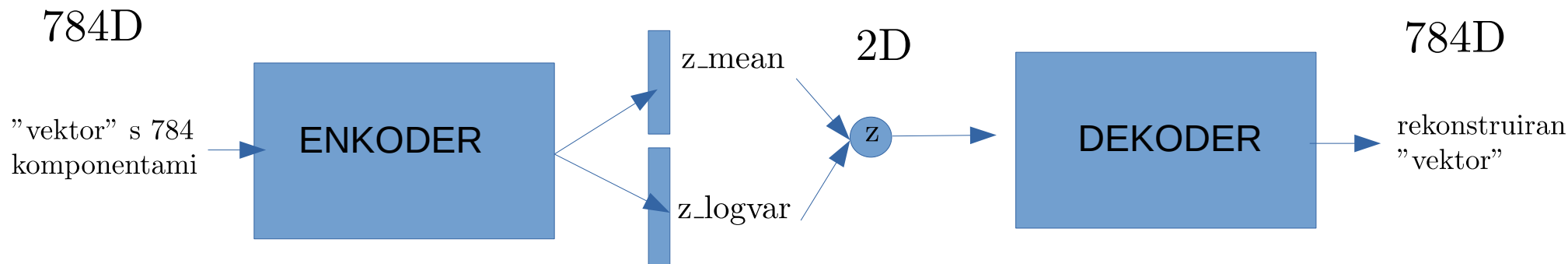
Normalizacija - vrednosti med 0 in 1

```
x_train = x_train/255
```



2. del: VAE + MNIST

Shema VAE



$$z = z_mean + \exp(z_logvar/2) \cdot \epsilon$$
$$\epsilon \sim N(\mu = 0, \sigma = 1)$$

2. del: VAE + MNIST

9/29

Implementacija enkoderja (Keras)

```
input_layer = keras.Input(shape=(28*28,))  
hidden_layer = keras.layers.Dense(64, activation = 'selu')(input_layer)  
z_mean = keras.layers.Dense(2, activation = None)(hidden_layer)  
z_log_var = keras.layers.Dense(2, activation = None)(hidden_layer)
```

1 skrita plast s 64 neuroni

dimenzija latentnega prostora

Vzorčenje

```
from keras import backend as K  
  
def sampling(args):  
    z_mean, z_log_var = args  
    epsilon = K.random_normal(shape = K.shape(z_mean))  
    return z_mean + tf.exp(0.5 * z_log_var) * epsilon
```

$$z = z_mean + \exp(z_logvar/2) \cdot \epsilon$$
$$\epsilon \sim N(\mu = 0, \sigma = 1)$$

```
z = layers.Lambda(sampling)([z_mean, z_log_var])
```

```
encoder = keras.Model(E_input_layer, [z_mean, z_log_var, z])
```

2. del: VAE + MNIST

10/29

Implementacija dekoderja (Keras)

1 skrivna plast s 64 neuroni

```
D_input_layer = keras.Input(shape=(2,))  
D_hidden_layer = keras.layers.Dense(64, activation = 'selu')(D_input_layer)  
D_output_layer = keras.layers.Dense(784, activation = 'sigmoid')(D_hidden_layer)  
decoder = keras.Model(D_input_layer, D_output_layer)
```

Izhod enkoderja

[z_mean, z_logvar, z]
0 1 2

želimo vednosti v intervalu [0,1]

```
outputs = decoder(encoder(input_layer)[2])  
VAE = keras.Model(input_layer, outputs, name = 'VAE')
```

vhodna plast enkoderja

izhodna plast dekoderja

2. del: VAE + MNIST

11/29

Cenovna funkcija

minimiziramo $\mathcal{L} = D(x, x') + KL(z_{\text{mean}}, e^{\frac{1}{2} z_{\text{logvar}}} | 0, 1)$

x vhodni "vektor"
 x' rekonstruiran "vektor"

Mera med vektorja, želimo,
da sta si x in x' čim bolj podobna

Kullback-Leibler divergenca

Želimo, da so točke v latentnem prostoru čim bližje skupaj
(želimo, da je porazdelitev z -jev čim bolj podobna $N(0, 1)$)

+ dobra rekonstrukcija
- točke niso nujno blizu skupaj

trade-off

+ točke blizu skupaj
- slabša rekonstrukcija "vektorjev"

2. del: VAE + MNIST

Cenovna funkcija in optimizer

$$\mathcal{L} = D(x, x') + \beta \cdot KL(z_{\text{mean}}, e^{\frac{1}{2} z_{\text{logvar}}} | 0, 1)$$

Včasih je smiselno spremeniti vpliv KL divergence

Implementacija cenovne funkcije

```
rec_loss = keras.losses.binary_crossentropy(input_layer, outputs) * 28*28
kl_loss = - 0.5 * K.sum(1 + z_log_var - K.square(z_mean) - K.exp(z_log_var))
vae_loss = K.mean(rec_loss + kl_loss)
VAE.add_loss(vae_loss)
VAE.compile(optimizer = 'adam')
```

$1/\beta$

2. del: VAE + MNIST

Učenje modela

```
history = VAE.fit(x = x_train,  
                 y = x_train,  
                 batch_size = 32,  
                 epochs = 100)
```

(ne, ni napaka)

```
history.history
```

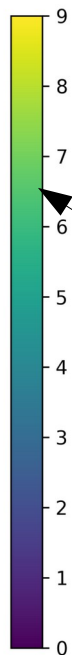
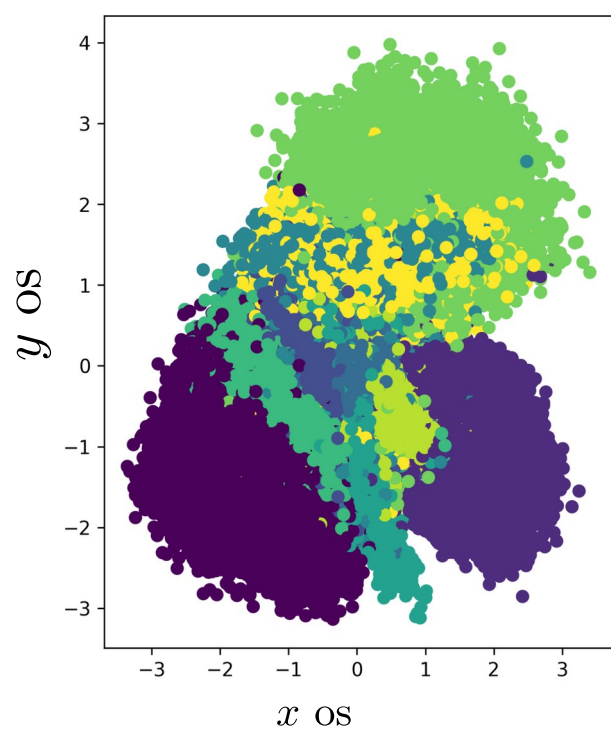
vsebuje vrednosti cenovne funkcije za vsak epoch

2. del: VAE + MNIST

3. naloga

Nariši točke v latentnem prostoru za vse števke.

2D latentni prostor



Ta legenda ni ok

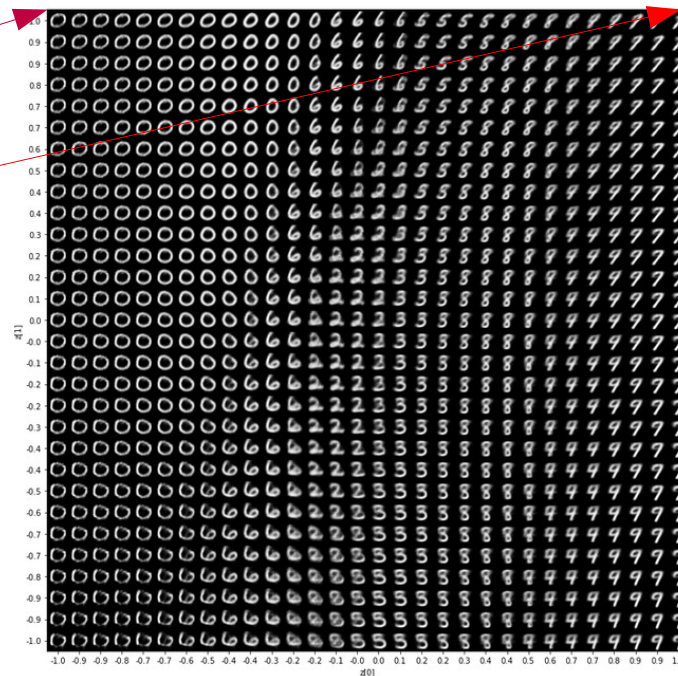
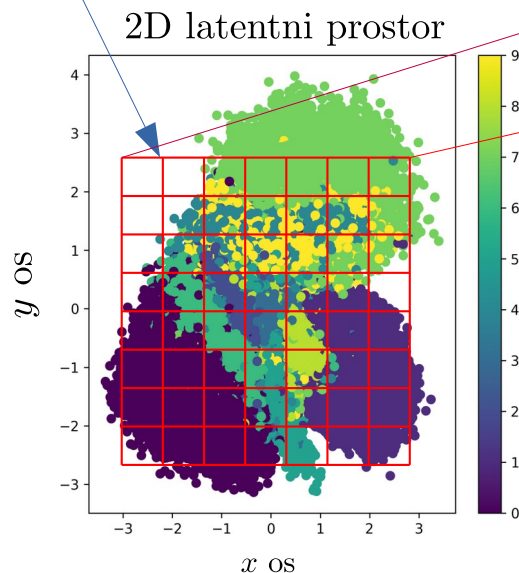
```
z_train_mean = encoder.predict(x_train)[0]  
z_test_mean = encoder.predict(x_test)[0]
```

2. del: VAE + MNIST

4. naloga

Generiraj slike iz latentnega prostora tako, da najprej določiš mrežo, ki dobro pokriva območje točk v latentnem prostoru in nato dekodiraš točke, ki jih mreža določa. Primer:

recimo mreža 15×15



Vzeto iz: <https://keras.io/examples/generative/vae/>

3. del: Iskanje dvocurkovnega signala

16/29

Standardni model (SM) odlično opisuje fiziko delcev.

razen nevtrinov (v SM so brezmasni, eksperimenti
pa so implicitno pokazali, da imajo neničelno maso)

Obstajajo eksperimentalne meritve, ki morda nakazujejo fiziko izven SM.

3. del: Iskanje dvocurkovnega signala

17/29

- Obstaja mnogo razširitev SM (z in brez supersimetrije), ki skušajo razrešiti določene težave SM.
- Običajno se išče novo fiziko na podlagi napovedi izbranega modela....
- Metode iskanja nove fizike, ki so neodvisne od izbire modelov se trenutno intenzivno raziskuje - v ta namen je 2. del naloge posvečen uporabi VAE za iskanje nove fizike.

3. del: Iskanje dvocurkovnega signala

18/29

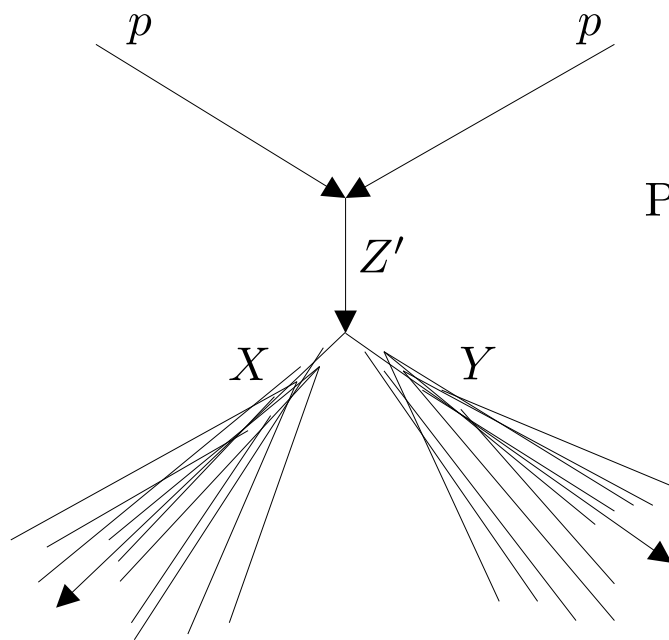
Dvocurkovni signal

Želimo določiti mase delcev X, Y, Z'
fiktivnega procesa $p p \rightarrow Z' \rightarrow X Y$

Podatki vsebujejo opazljivke dveh tipov dogodkov:

→ Dogodki signala
 $p p \rightarrow Z' \rightarrow X Y$

→ Dogodki ozadja
 $p p \rightarrow$ delci iz standardnega modela



$$m_{Z'} = 3.5 \text{ TeV}$$

$$m_X = 500 \text{ GeV}$$

$$m_Y = 100 \text{ GeV}$$

Uporabimo VAE za nenadzorovano klasifikacijo dogodkov.
VAE preiskusimo na podatkih z znanimi oznakami dogodkov,
nato pa ga uporabimo na podatkih "črne škatle" (black box).

3. del: Iskanje dvocurkovnega signala

19/29

Plan:

→ VAE z dimenzijo latentnega prostora 1 učimo na vseh podatkih

→ Konstruiramo klasifikator v latentnem prostoru tako, da vemo kje se nahajajo podatki signala

→ Uporabimo klasifikator, da določimo množico podatkov, ki vsebujejo največji delež dogodkov signala. S temi določimo mase delcev .

3. del: Iskanje dvocurkovnega signala

20/29

Testni podatki:

lhco_L_labels.npy
lhco_H_labels.npy

Oznake dogodkov (samo za grafe)

`np.array([0, 0, 1, 1, 0, 0, ...])`

0 : ozadje

1 : signal

lhco_L_jetobs.npy
lhco_H_jetobs.npy

8 spremenljivk na dogodek (VAE učimo s temi podatki)

`*.shape = ( $\sim 10^6$ , 8)`

$\{[m_{j_1}, (\tau_2/\tau_1)_1, (\tau_3/\tau_2)_1, m_{d_{j_1}}], [m_{j_2}, (\tau_2/\tau_1)_2, (\tau_3/\tau_2)_2, m_{d_{j_2}}]\}$

1. curek

2. curek

L : $S/B = 0.001$

H : $S/B = 0.1$

Razmerje med dogodki signala (S) in ozadja (B):

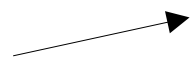
3. del: Iskanje dvocurkovnega signala

21/29

Testni podatki:

lhco_L_invmass.npy
lhco_H_invmass.npy

Invariantne mase dogodkov
(samo za grafe in analizo)
1D numpy array



Podatki "črne škatle":

blackbox_jetobs.npy

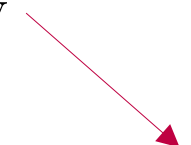
blackbox_invmass.npy

8 spremenljivk na dogodek (VAE učimo s temi podatki)

$\{[m_{j_1}, (\tau_2/\tau_1)_1, (\tau_3/\tau_2)_1, m_{d_{j_1}}], [m_{j_2}, (\tau_2/\tau_1)_2, (\tau_3/\tau_2)_2, m_{d_{j_2}}]\}$

1. curek

2. curek



Invariantne mase dogodkov

L : $S/B = 0.001$

H : $S/B = 0.1$

Razmerje med dogodki signala (S) in ozadja (B):

3. del: Iskanje dvocurkovnega signala

22/29

VAE treniramo na "vektorjih":

$$\{[m_{j_1}, (\tau_2/\tau_1)_1, (\tau_3/\tau_2)_1, m_{d_{j_1}}, m_{j_2}, (\tau_2/\tau_1)_2, (\tau_3/\tau_2)_2, m_{d_{j_2}}]\}$$

Izkaže se, da so rezultati boljši, če najprej transformiramo vhodne podatke na naslednji način:

```
from sklearn.preprocessing import QuantileTransformer

x_train = np.load(x_path)
x_train = x_train.astype('float32')
f_scaler = QuantileTransformer(output_distribution='uniform')
x_train_transformed = f_scaler.fit_transform(x_train)
```

Porazdelitev podatkov spremeni v enakomerno

Inverzna preslikava:

```
f_scaler.inverse_transform(X)
```

3. del: Iskanje dvocurkovnega signala

23/29

Hiperparametri VAE

- Enkoder: 3 skriti sloji, vsak z 64 neuroni
 - Dekoder: 3 skriti sloji, vsak z 64 neuroni
 - Med sloji aktivacijska funkcija "selu" (razen za zadnji sloj dekoderja in za sloja z_{mean} in z_{logvar})
 - batch_size = 1000, epochs = 100, dimenzija latentnega prostora 1
 - Cenovna funkcija: $5000 * \text{MSE}(x, x') + \text{KL}$
 - Optimizer "Adadelata"
- $1/\beta$

Treniraj VAE na transformiranih podatkih!

Rezultate prikaži s fizikalnimi podatki (transformiraj nazaj).

3. del: Iskanje dvocurkovnega signala

24/29

5. Nariši porazdelitve prvih 4 spremenljivk za signal, ozadje in oboje skupaj (4 grafi) za podatke z $S/B = 10\%$. Nato transformiraj te iste podatke z `QuantileTransformer` in tako kot prej nariši njihove porazdelitve (spet 4 grafi). Preveri še, da inverzna transformacija deluje.
6. Implementiraj VAE z ustrezni hiperparametri, glej poglavje [4.2](#), ter ga natreniraj z uporabo optimizerja `adadelta` na podatkih z $S/B = 10\%$ in $S/B = 0.1\%$ do epocha 100. Postopek ponovi vsaj 3-krat in shrani uteži modela.
7. Z enkoderjem naučenega VAE preslikaj vse dogodke v latentni prostor. Nariši porazdelitve signala in ozadja za spremenljivke z_{mean} , z_{logvar} in z_{mean}^2 za podatke z $S/B = 10\%$ in $S/B = 0.1\%$ (2 grafa za vsak naučen VAE). Kje se nahaja večina dogodkov signala? Pojasni ugotovitve (ta del je ključen pri reševanju nalog, ki uporabljajo podatke črne škatle).
8. Nariši ROC krivuljo za klasifikatorja z_{mean}^2 in z_{logvar} za $S/B = 10\%$ in $S/B = 0.1\%$ ter izračunaj AUC. Namig: S knjižnico *sklearn* se lahko uporabijo naslednji ukazi:

```
from sklearn import metrics
auc = metrics.roc_auc_score(labels, measure)
fpr, tpr, thresholds = metrics.roc_curve(labels, measure)
```

kjer je *measure* izbran klasifikator $\pm z_{\text{mean}}^2$ in $\pm z_{\text{logvar}}$ (predznak določa iz katere smeri začnemo rezati podatke, izberi ga tako, da bo AUC največji) Nazadnje nariši graf funkcije $1 / \text{fpr}$ (po ang. *inverse false positive rate*) v odvisnosti od *tpr* (ang. *true positive rate*).

3. del: Iskanje dvocurkovnega signala

25/29

9. Razvrsti netransformirane podatke za invariantno maso, m_{j1} in m_{j2} po naraščajoči ali padajoči vrednosti klasifikatorja z največjim AUC (glej prejšnjo točko) tako, da bo večina dogodkov signala na začetku seznama. Vzami prvih 500, 1000 in 2000 dogodkov iz tega seznama in poskusi določiti maso neznanega delca in masi obeh produktov z negotovostjo *Iz porazdelitev m_{j1} se lahko oceni masa in negotovost prvega curka, morda se splača omejiti dogodke na invariantne mase med 3.3 TeV in 3.7 TeV.*
10. Izberi 1 naučen VAE in preveri kako dobro rekonstruira porazdelitve prvih 4 spremenljivk (ne pozabi na inverzno transformacijo).
11. **Dodatna:** Treniraj še en VAE tako, da med učenjem izračunaš AUC ob koncu vsakega epoch-a. Nariši graf funkcije AUC.
12. **Dodatna:** Kako se rekonstrukcija in AUC klasifikatorjev spremeni, če neamesto `adadelta`-e uporabimo `adam` optimizer?
13. **Dodatno:** Kako se rekonstrukcija in AUC klasifikatorjev spremeni, če povečamo dimenzijo latentnega prostora?

3. del: Iskanje dvocurkovnega signala

26/29

Naloge, ki se nanašajo na podatke črne škatle:

14. Treniraj VAE na transformiranih podatkih črne škatle. Po učenju izračunaj z_{mean}^2 in z_{logvar} za vsak dogodek in shrani rezultate.
15. Poskušaj določiti maso delca in obeh produktov z ustreznimi negotovostmi tako, da slediš postopku iz 9. točke. Sklepaj kje se nahajajo podatki signala iz ugotovitev pridobljenih iz 7. naloge. *Namig: Morda se bo splačalo omejiti dogodke na invariantne mase med 3.6 TeV in 4.0 TeV.*

Dodatek za 9. in 14. nalogo 7. sklopa

Maso delcev Z' , X in Y se določi tako, da se nariše po navodilih 9. naloge porazdelitve za invariantno maso m_{inv} , m_{j1} in m_{j2} .

Vrednosti za m_{inv} , m_{j1} in m_{j2} pri kateri porazdelitve imajo vrh so pričakovane vrednosti mas delcev Z' , X in Y (v tem vrstnem redu).

Vsaka porazdelitev vsebuje nekaj dogodkov signala in nekaj ozadja.

Določanje negotovosti mas ni trivialno, ker ima porazdelitev ozadja tudi vrh (torej s fitanjem krivulje Crystall ball dobimo interval zaupanja, ki je večji od dejanskega - kar je ok).

Zaradi te komplikacije sprejemem tudi druge načine določanja negotovosti, vendar z razlago kaj dobljen interval zaupanja pomeni.

Ocenjevanje za 6. in 7. sklop nalog

- Posamezna naloga je vredna 1 točko
- Posamezna dodatna naloga je tudi vredna 1 točko
- Za oceno 10 je dovolj rešiti vse (ne dodatne) naloge po navodilih
- Dodatne grafe in izračune se upošteva pri končni ocene (zaokroževanje)

Rok za oddajo poročila:
28.2.2026, 23:59:59

Kriterij:

- a) vsaka naloga je vredna 1 točko
- b) najmanjša resolucija 1/8 točke
- c) $\geq 90\%$ točk ocena 10 (nič ekstra ni potrebno)
- d) zamude vplivajo na zaokroževanje končne ocene predmeta, če je ta medoceno

Rok za oddajo (6. in 7. sklop): 28.2.2026, 23:59:59

Poročilo oddaj na:

blaz.bortolato.reports@proton.me

s pripisom: PSUF-7.sklop